

SkillTrack

система управления учебными IT-проектами, задачами и портфолио

Срок выполнения	Формат	Итог
04.05-13.05.2026	1 задание Web + 1 задание Mobile	2 победителя: Web и Mobile

Главная идея

Студент за короткий срок с помощью ИИ-ассистентов реализует максимально законченный, рабочий и аккуратно оформленный проект. Побеждает не самый большой по количеству строк код, а самый функциональный, стабильный и понятный продукт.

1. Общая логика конкурса

Конкурс проходит в формате короткого продуктового спринта. Каждый студент выбирает одно из двух направлений - Web или Mobile - и реализует проект по общей теме SkillTrack. Использование ИИ разрешено и приветствуется, но студент несет ответственность за работоспособность, архитектуру и понимание собственного кода.

Параметр	Решение
Период	04.05-13.05.2026
Формат	индивидуальная работа; стек студент выбирает самостоятельно
Победители	1 лучшая работа по Web и 1 лучшая работа по Mobile
Главный критерий	объем реально работающего функционала при сохранении чистоты кода и нормальной БД
Минимальный результат	проект запускается локально, имеет README, демо-данные и понятный пользовательский сценарий
Запрещено	сдать набор статичных экранов без логики как разработческую работу; выдать неработающий AI-код за готовый проект

Как понимать «победа по объему»

Объем - это количество завершенных и проверяемых функций: создано, сохраняется, редактируется, удаляется, фильтруется, валидируется, отображается и не ломается. Количество строк кода, количество экранов без логики и визуальный шум не считаются объемом.

2. Единая тема проектов SkillTrack

SkillTrack - это система, которая помогает студенту и ментору вести учебные IT-проекты: создавать проекты, ставить задачи, отслеживать прогресс, фиксировать навыки, собирать портфолио и готовиться к практике или стажировке.

Блок	Описание
Пользовательская проблема	Студенты часто делают учебные проекты хаотично: задачи не описаны, прогресс не виден, портфолио не собрано, дедлайны теряются.
Продуктовая идея	Сделать компактный трекер учебных проектов, задач, навыков и достижений.
Почему тема универсальна	Подходит и для веб-приложения, и для мобильного приложения; не требует сложной доменной экспертизы; легко расширяется.
Что можно показать	CRUD, роли, фильтры, статусы, календарь, комментарии, локальную БД, REST API, адаптивный интерфейс, авторизацию, аналитику.

3. Задание для направления Web

ТЕМА WEB-ПРОЕКТА

SkillTrack Web - веб-платформа для управления учебными IT-проектами, задачами, навыками студентов и портфолио.

Web-работа должна выглядеть как небольшая CRM/панель управления: пользователь заходит в систему, видит проекты, задачи, статусы, дедлайны, участников, навыки и прогресс. В идеале проект должен иметь backend, базу данных и frontend. Если студент работает только как frontend-разработчик, он может

использовать mock API или локальный JSON-server, но за полноценную БД и backend начисляется больше баллов.

3.1. Рекомендуемый стек

Слой	Допустимые варианты
Frontend	Vue 3 / Nuxt / React / чистый HTML+CSS+JS - стек свободный, но интерфейс должен быть аккуратным и адаптивным.
Backend	Laravel / Nest.js / Django / Express - любой понятный студенту backend.
База данных	PostgreSQL / MySQL. Важно наличие нормальной структуры, миграций или SQL-скриптов.
Инфраструктура	Docker (не обязательно, но будет плюсом), seeders, .env.example, README, Swagger/OpenAPI - дополнительные плюсы.

3.2. Обязательное ядро функциональности

Функция	Что должно работать
Авторизация	Регистрация/вход/выход или хотя бы демо-вход по ролям. Пароли не хранить в открытом виде, если есть backend.
Dashboard	Главная панель с цифрами: количество проектов, задач, просроченных задач, завершенных задач, общий прогресс.
Проекты	CRUD проектов: название, описание, статус, дедлайн, участники, стек/навыки.
Задачи	CRUD задач внутри проекта: статус, приоритет, дедлайн, ответственный, описание.
Статусы	Минимум 4 статуса: backlog / in progress / review / done. Можно реализовать Kanban или таблицу.
Фильтры и поиск	Поиск по названию, фильтрация по статусу/приоритету/дедлайну/участнику.
Профиль студента	Имя, направление, навыки, уровень навыка, ссылки на GitHub/портфолио.
Валидация	Проверка обязательных полей, корректная обработка ошибок и пустых состояний.
Дизайн	Единая визуальная система: сетка, карточки, таблицы, кнопки, состояния, адаптивность.
БД	Минимум 4 связанные сущности: users, projects, tasks, skills или comments.

3.3. Функции для сильных студентов

Расширение	Как засчитывается
Роли	admin/mentor/student: разные права доступа, разные меню и разные действия.
Комментарии	Комментарии к задачам или проектам с автором и датой.
История действий	Лог активности: кто создал, изменил, закрыл задачу.
Файлы	Прикрепление ссылок или файлов к проекту/задаче.
Kanban drag-and-drop	Перетаскивание задач между статусами.
Календарь	Отображение дедлайнов по дням или список ближайших сроков.
Экспорт	CSV/PDF/Excel выгрузка списка задач или проектов.
API-документация	Swagger/OpenAPI или отдельный README с описанием endpoints.
Тесты	Минимум несколько unit/feature/e2e-тестов.
Docker	docker-compose для быстрого запуска проекта и базы данных.
Деплой	Публичная ссылка на демо или видео с локальным запуском.

3.4. Минимальная структура страниц

Страница	Содержание
/login	вход или демо-вход
/dashboard	метрики, ближайшие дедлайны, быстрые действия
/projects	список проектов, поиск, фильтры, создание
/projects/{id}	карточка проекта, задачи, участники, прогресс
/tasks	общий список или Kanban задач
/profile	профиль студента, навыки, ссылки
/settings	настройки темы/профиля - опционально

Для веб-дизайнеров

Если участник идет именно как веб-дизайнер, допустима сдача уникального UI-дизайна SkillTrack: 6-8 экранов, UI-kit, компоненты, адаптивы и кликабельный прототип. Но в общем зачете разработчиков приоритет остается за рабочей реализацией с функционалом.

4. Задание для направления Mobile

ТЕМА MOBILE-ПРОЕКТА

SkillTrack Mobile - мобильное приложение студента для ведения учебных IT-проектов, задач, дедлайнов, навыков и личного портфолио.

Mobile-работа должна быть самостоятельным приложением, которое можно запустить на эмуляторе или устройстве. Приложение может работать полностью локально или через mock/API. Важно, чтобы данные сохранялись, интерфейс был аккуратным, а навигация и состояния приложения были понятными.

4.1. Рекомендуемый стек

Слой	Допустимые варианты
Основной вариант	Flutter + Dart.
State management	Bloc / Riverpod / Provider / setState - допускается любой вариант, но архитектура должна быть понятной.
Сеть	Dio / http. Можно подключить mock API, JSON-server, Firebase/Supabase или свой backend.
Локальное хранилище	sqflite / Drift / Hive / Isar. SharedPreferences можно использовать только для настроек, но не как основную БД.
Инструменты	Android Studio / VS Code / Git / Postman / Figma.

4.2. Обязательное ядро функциональности

Функция	Что должно работать
Onboarding	1-3 стартовых экрана или welcome screen с объяснением продукта.
Демо-вход	Экран входа: можно без настоящего сервера, но с сохранением состояния пользователя.
Главный экран	Список проектов, общий прогресс, ближайшие задачи, быстрые действия.
Проекты	Создание, просмотр, редактирование и удаление учебного проекта.
Задачи	Создание задач внутри проекта, смена статуса, приоритет, дедлайн.
Поиск/фильтры	Фильтрация задач по статусу, приоритету или дедлайну; поиск по названию.
Профиль	Имя, направление, навыки, ссылки, краткое описание.
Локальная БД	Данные не должны пропадать после закрытия приложения.
Пустые состояния	Если нет проектов/задач, приложение должно показывать понятный экран, а не пустую страницу.

Функция	Что должно работать
Дизайн	Единый стиль: карточки, отступы, цвета, типографика, адекватная работа на разных размерах экрана.

4.3. Функции для сильных студентов

Расширение	Как засчитывается
Синхронизация	Получение/отправка данных через API или mock backend.
Offline-first	Работа без сети с последующей синхронизацией.
Уведомления	Локальные уведомления о дедлайнах.
Статистика	График или диаграмма по выполненным задачам/проектам.
Темная тема	Переключение темы и сохранение настройки.
Архитектура	Разделение data/domain/presentation, repositories, use cases.
State management	Осознанное использование Bloc/Riverpod с понятной структурой состояний.
Тесты	Unit/widget tests для ключевой логики.
Сборка	APK/AAB или понятная инструкция запуска.
Публикационный опыт	README с описанием подготовки сборки и подписи - дополнительный плюс.

4.4. Минимальная структура экранов

Экран	Содержание
Splash / Onboarding	первый запуск, краткое объяснение приложения
Login	демо-вход или локальный профиль
Home	проекты, прогресс, ближайшие задачи
Projects	список проектов, создание, поиск
Project Details	описание проекта, задачи, статистика
Task Form	создание/редактирование задачи
Profile	навыки, ссылки, личные данные
Settings	тема, очистка демо-данных, информация о приложении

5. Правила использования ИИ

ИИ-инструменты разрешены: Claude Code, Codex, Qwen, Gemini и другие помощники. При этом студент должен уметь объяснить, что делает его проект, как устроена БД, как запустить приложение и где находится ключевая логика.

Правило	Содержание
Разрешено	генерировать код, компоненты, миграции, тесты, README, идеи дизайна, исправления ошибок
Обязательно	проверять, запускать, чистить, комментировать и понимать полученный код
Запрещено	сдавать неработающий код, хранить реальные токены, подключать боевые базы, копировать чужой проект без переработки
Желательно	оставить в README короткий блок: какие ИИ-инструменты использовались и для каких задач

Практический стандарт

ИИ должен ускорять разработку, но не заменять ответственность разработчика. Если проект не запускается, нет структуры БД, README пустой, а студент не может объяснить код - работа не должна занимать призовое место, даже если визуально выглядит красиво.

6. Критерии оценки и выбор победителей

Оценка проводится отдельно по направлениям Web и Mobile. Максимум - 100 баллов. Победителем становится участник с наибольшим количеством баллов, при условии что проект запускается и демонстрирует работоспособный функционал.

Критерий	Баллы	Что проверяется
Функциональность	35	Количество и завершенность рабочих функций: CRUD, статусы, фильтры, поиск, роли, комментарии, статистика, экспорт и т.д.
Дизайн и UX	15	Единый стиль, аккуратные отступы, понятная навигация, адаптивность, пустые/ошибочные состояния.
База данных и данные	15	Нормальная структура сущностей, связи, миграции/seeder, локальная БД для mobile, отсутствие хаотичного хранения.
Качество кода	20	Структура проекта, понятные компоненты/сервисы, комментарии в сложных местах, отсутствие явных костылей.
Стабильность	10	Проект запускается, не падает на базовых сценариях, есть демо-данные, обработаны ошибки.
Презентация	5	README, инструкция запуска, демо-логины, скриншоты или короткое видео.

6.1. Автоматические основания для отклонения

- проект не запускается и нет понятного способа проверить результат;
- нет исходного кода или репозиторий пустой;
- нет README с инструкцией запуска;
- для разработческой работы сдан только дизайн без работающей логики;
- основной функционал не сохраняет данные;
- в репозитории лежат реальные секреты, токены, пароли или приватные ключи;
- явно скопирован чужой проект без адаптации под тему SkillTrack.

7. Требования к сдаче работы

Каждый студент должен сдать результат так, чтобы ментор мог быстро открыть, запустить и проверить проект без долгих уточнений.

Что сдать	Требование
GitHub/GitLab	ссылка на репозиторий с актуальным кодом
README.md	текст, описание проекта, инструкция запуска, демо-логин, структура БД, список реализованных функций
.env.example	пример переменных окружения без реальных секретов
БД	миграции/seeder/SQL-дамп или инструкция создания локальной БД
Демо	короткое видео 2-5 минут или набор скриншотов с основными сценариями
Web	желательно ссылка на deploy; если деплоя нет - четкая инструкция локального запуска
Mobile	APK/debug build или инструкция запуска через Flutter/Android Studio

8. План выполнения с 04.05 по 13.05

Дата	Фокус	Результат дня
04.05	Проектирование	Выбрать стек, создать репозиторий, описать сущности, набросать дизайн, подготовить БД и структуру проекта.
05.05	Базовая реализация	Сделать авторизацию/демо-вход, проекты, задачи, базовые экраны, подключить хранение данных.
06.05	Расширение функционала	Добавить фильтры, поиск, статусы, профиль, комментарии, dashboard/статистику.
07.05	Полировка	Улучшить дизайн, обработать ошибки, почистить код, добавить демо-данные, написать README.
13.05	Сдача	Записать короткое демо, проверить запуск с нуля, отправить ссылку на репозиторий и материалы.

9. Чек-листы проверки

Чек-лист студента перед сдачей

- ☐ Проект запускается с чистой установки по README.
- ☐ В базе есть демо-данные или есть понятная инструкция, как их создать.
- ☐ Все кнопки, формы, фильтры и CRUD-действия работают без критических ошибок.
- ☐ Нет реальных секретов в репозитории.
- ☐ Интерфейс выглядит цельно, нет хаотичных цветов и разных размеров шрифта без системы.
- ☐ Код разбит на компоненты/сервисы/слои, а не написан одним большим файлом.
- ☐ Сложные места кода прокомментированы кратко и по делу.
- ☐ В README перечислены реализованные функции и функции, которые не успел сделать.

Приложение 1. Рекомендуемая структура БД для Web

Таблица	Поля	Назначение
users	id, name, email, password_hash, role, direction, created_at	Пользователи: студент, ментор, администратор.
skills	id, name, category	Справочник навыков: Vue, Laravel, Flutter, SQL и т.д.
user_skills	user_id, skill_id, level	Связь пользователя и навыка с уровнем.
projects	id, title, description, status, deadline, owner_id	Учебные проекты.
project_members	project_id, user_id, project_role	Участники проекта.
tasks	id, project_id, title, description, status, priority, deadline, assignee_id, position	Задачи проекта.
comments	id, task_id, user_id, body, created_at	Комментарии к задачам.
activity_logs	id, user_id, entity_type, entity_id, action, created_at	История действий - опционально.

Приложение 2. Рекомендуемая локальная структура для Mobile

Таблица/коллекция	Поля	Назначение
profile	id, name, direction, about, github_url	Локальный профиль пользователя.

HRACE

Таблица/коллекция	Поля	Назначение
projects	id, title, description, status, deadline, progress	Список проектов.
tasks	id, project_id, title, status, priority, deadline, is_done	Задачи внутри проекта.
skills	id, name, level	Навыки студента.
notes	id, project_id, body, created_at	Заметки по проектам - опционально.
settings	key, value	Тема, состояние onboarding, настройки приложения.

Приложение 3. Идеи расширений для победы по объему

- роли и права доступа;
- drag-and-drop Kanban;
- статистика прогресса по проектам;
- экспорт задач;
- комментарии и история изменений;
- файлы или ссылки на материалы;
- темная тема;
- локальные уведомления в mobile;
- API-документация;
- тесты и CI;
- Docker-окружение;
- короткое демо-видео и качественный README.

Приложение 4. Стартовые промпты для Claude Code / Codex

Эти промпты не заменяют разработку, а помогают быстро разложить задачу на архитектуру и этапы.

Сценарий	Промпт
Планирование	Проанализируй это ТЗ и предложи архитектуру проекта, структуру папок, сущности БД и план разработки на 5 дней. Не пиши код, сначала дай план.
База данных	Сгенерируй миграции/SQL-схему для сущностей SkillTrack: users, projects, tasks, skills, comments. Учти связи, индексы, статусы и seed-данные.
Frontend	Собери дизайн-систему для SkillTrack: цвета, типографика, компоненты карточек, таблиц, форм, пустых состояний и dashboard.
Backend	Реализуй CRUD для проектов и задач с валидацией, обработкой ошибок и понятной структурой сервисов/контроллеров.
Mobile	Спроектируй Flutter-приложение SkillTrack с локальной БД, слоями data/domain/presentation и экранами Home, Projects, Project Details, Task Form, Profile.
Проверка	Проверь проект на ошибки запуска, дублирование кода, неиспользуемые файлы, небезопасные секреты и проблемы в README.

Официальные страницы: [Claude Code Overview](#); [OpenAI Codex CLI](#)